Distribution Management System

Headless Server Hosting and Tailscale Access Runbook

Document version: 1.0
Date: 2026-04-07

---

1. Purpose

This runbook provides a complete, production-oriented deployment procedure for hosting the Distribution Management System on a Linux server without GUI and granting secure access through Tailscale.

It covers:

- Server provisioning and hardening
- Application deployment with Docker Compose
- Configuration required by this specific codebase
- Private access model using Tailscale
- How to add technical users (tailnet access)
- How to add application users (system login accounts)
- Operations, backup, and troubleshooting

---

2. System-Specific Findings (Validated Against Current Codebase)

These points are critical and specific to this repository:

1. Runtime ports:

- Frontend: 5173
- Backend API: 8080
- MySQL: 3306

2. Backend stack and DB:

- FastAPI backend
- MySQL via aiomysql
- Database schema auto-initialized by backend startup logic

3. Seeded initial account behavior:

- On first startup, backend seeds super admin `admin@dms.com`
- Initial password comes from `ADMIN_INITIAL_PASSWORD` (if set), else fallback default is used
- First login forces email and password change

4. Auth/session model:

- Cookie-based auth (`access_token`, `refresh_token`) with CSRF protection
- In production mode, auth cookies are `Secure`

- Therefore production access should be HTTPS to avoid cookie/login issues

5. Frontend API URL behavior:

- Frontend uses `VITE_API_URL` at build time
- If not explicitly set for production build, fallback is `http://localhost:8080/api` (incorrect for remote users)
- DevOps must set a production value before building frontend image

6. Docker compose topology:

- `mysql`, `backend`, `frontend` services
- Persistent MySQL volume (`mysql_data`)
- Backend bind mounts for manifests, backups, uploads

---

3. Target Architecture (Headless + Private Access)

Recommended model:

- Linux server (no GUI) runs Docker Compose stack.
- Server joins tailnet using Tailscale.
- Users access app only through tailnet (private network).
- HTTPS termination is done via Tailscale Serve.
- App URL used by users: `https://<server-hostname>.<tailnet>.ts.net`

Traffic flow:

1. User device joins tailnet (authenticated user).
2. User opens app URL on tailnet HTTPS endpoint.
3. Tailscale Serve forwards traffic to frontend container (port 5173).
4. Frontend calls backend using configured API URL on same tailnet hostname.

---

4. Prerequisites

- Ubuntu 22.04/24.04 LTS server (recommended)
- Sudo access
- Outbound internet from server for package pulls
- Docker Engine + Docker Compose plugin
- Tailscale account and tailnet admin privileges
- DNS/MagicDNS enabled in Tailscale admin console

---

5. Server Preparation (No GUI)

Run as a sudo-capable user.

```
sudo apt update && sudo apt -y upgrade
sudo timedatectl set-timezone UTC
```

```
sudo apt -y install ca-certificates curl gnupg lsb-release git ufw jq
```

Optional but recommended firewall baseline:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw enable
sudo ufw status verbose
```

Note: Do not expose app ports publicly. Access should occur through Tailscale.

---

6. Install Docker and Compose Plugin

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt -y install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

Log out and log back in once so docker group membership applies.

---

7. Install and Join Tailscale

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

After running `tailscale up`, complete browser/device authorization with a tailnet admin.

Validate:

```
tailscale status
tailscale ip -4
tailscale ip -6
```

---

8. Deploy Application Code

```
sudo mkdir -p /opt/dms
sudo chown $USER:$USER /opt/dms
cd /opt/dms
git clone <your-repo-url> distribution-management-system
cd distribution-management-system
```

Use the required branch/tag:

```
git checkout <release-branch-or-tag>
```

---

9. Configure Environment for This Project

9.1 Backend environment file

Create `backend/.env`:

```
# Application
APP_NAME=Distribution Management System
APP_VERSION=1.0.0
DEBUG=False
ENVIRONMENT=production

# Server
HOST=0.0.0.0
PORT=8080

# Database
DB_HOST=mysql
DB_PORT=3306
DB_USER=dms_user
DB_PASSWORD=<strong-random-db-password>
DB_NAME=distribution_management_system

# Security
SECRET_KEY=<generate-64-byte-urlsafe-secret>
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7
```

```
CSRF_COOKIE_SECURE=true
ENFORCE_HTTPS=false

# CORS (allow tailnet app origin)
CORS_ORIGINS=https://<server-hostname>.<tailnet>.ts.net

# Seeded super admin initial password
ADMIN_INITIAL_PASSWORD=<strong-temporary-admin-password>
```

Generate a strong secret key:

```
python3 - << 'PY'
import secrets
print(secrets.token_urlsafe(64))
PY
```

9.2 Frontend production API URL

This is mandatory for this repository because frontend reads `VITE_API_URL` at build time.

Create `frontend/.env.production`:

```
VITE_API_URL=https://<server-hostname>.<tailnet>.ts.net/api
```

Important:

- If this file is missing, frontend may be built with `http://localhost:8080/api`, which breaks remote access.

---

10. Optional Production Compose Override (Recommended)

The current base compose maps MySQL to host port 3306. For private production deployments, remove host exposure.

Create `docker-compose.prod.yml`:

```
services:
  mysql:
    ports: []
```

This keeps MySQL reachable only within Docker network.

---

11. Start Services

Without override:

```
docker compose up -d --build
```

With recommended override:

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

Check health:

```
docker compose ps
docker compose logs backend --tail=100
docker compose logs frontend --tail=100
docker compose logs mysql --tail=100
```

Validate local service endpoints from server shell:

```
curl -sS http://127.0.0.1:8080/health
curl -I http://127.0.0.1:5173
```

---

12. Publish HTTPS via Tailscale Serve (Headless-Friendly)

Expose frontend securely on tailnet HTTPS:

```
sudo tailscale serve --https=443 / http://127.0.0.1:5173
sudo tailscale serve status
```

Result:

- Users access `https://<server-hostname>.<tailnet>.ts.net`
- TLS is handled by Tailscale
- Backend API remains internal, reached via frontend at `/api`

Persist serve config across reboots:

```
sudo systemctl enable --now tailscaled
```

Note:

- Tailscale Serve configuration is stored by tailscaled state. Re-apply `tailscale serve` only if configuration is reset.

---

13. Access for End Users (How They Reach the App)

Each end user must:

1. Have a Tailscale account approved in your tailnet.
2. Install Tailscale client on their device.
3. Sign in to Tailscale and confirm they are connected.
4. Open browser URL:

```
https://<server-hostname>.<tailnet>.ts.net
```

No public IP or VPN gateway exposure is required.

---

14. How to Add Technical Access (Tailnet Users)

This controls who can reach the server/network.

1. In Tailscale admin console, invite user by email.
2. Assign group/ACL role (example: `devops`, `dms-users`).
3. Ensure ACL allows destination node and port 443.
4. User logs in to Tailscale and verifies connectivity.

Example ACL concept (pseudo-policy):

- Allow `group:dms-users` to access `tag:dms-server:443`
- Allow `group:devops` to access `tag:dms-server:*` and Tailscale SSH

Recommended:

- Tag this server as `tag:dms-server`
- Use least privilege ACLs
- Enable device approval if required by policy

---

15. How to Add Application Users (Inside DMS)

This controls who can log in to the software itself.

15.1 First login

1. Login with seeded super admin account:

- Email: `admin@dms.com`
- Password: value set in `ADMIN_INITIAL_PASSWORD`

2. System will force credential update (email + password). Complete this immediately.

15.2 Create users from UI

1. Login as super admin.
2. Go to Users management screen.
3. Create users with required role and parent relationships.
4. Share initial credentials with users securely.

15.3 Create users by API (optional)

Endpoint:

```
POST /api/users
```

Required payload fields:

- `email`
- `name`
- `role`
- `password` (must include upper/lower/digit/special)
- `parent_id` for roles that require hierarchy placement

Role hierarchy in current implementation:

- super_admin
- md_director
- manager
- pdic_staff
- sub_distribution_manager
- sub_distributor
- cluster
- operator

Notes:

- Only authorized creator roles can create specific target roles.
- Parent-child assignment is validated by backend rules.

---

16. Day-2 Operations

16.1 Common commands

```
cd /opt/dms/distribution-management-system
docker compose ps
docker compose logs -f backend
docker compose restart backend
```

```
docker compose restart frontend
docker compose down
docker compose up -d
```

16.2 Update deployment

```
cd /opt/dms/distribution-management-system
git fetch --all
git checkout <release-branch-or-tag>
git pull --ff-only

docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

16.3 Backups

Data locations in this project:

- MySQL data volume: Docker volume `mysql_data`
- Backend backup folder: `backend/monthly_backups`
- Upload artifacts: `backend/uploads`
- Distribution manifests: `backend/distribution_manifests`

Recommended backup policy:

- Daily MySQL dump
- Daily filesystem backup for mounted backend folders
- Retention policy (for example 30/90 days)
- Periodic restore test

---

17. Validation Checklist (Go-Live)

- ☐ `docker compose ps` shows all services healthy/running
- ☐ `curl http://127.0.0.1:8080/health` returns healthy response
- ☐ Tailscale status is connected
- ☐ `tailscale serve status` shows HTTPS forwarding to port 5173
- ☐ Browser login works from a separate tailnet client
- ☐ Super admin forced credential update works
- ☐ New user creation works for at least 2 different roles
- ☐ API calls from frontend succeed (no CORS/cookie errors)
- ☐ Backups are generated and recoverable

---

18. Troubleshooting

1. Users can open app but cannot log in:

- Confirm `ENVIRONMENT=production`

- Confirm access URL is HTTPS via tailscale serve
- Confirm browser is using the exact configured tailnet hostname

2. Frontend cannot call API:

- Verify `frontend/.env.production` contains correct `VITE_API_URL`
- Rebuild frontend image after changing env file
- Check backend reachable at `http://127.0.0.1:8080/health` on server

3. CORS errors:

- Ensure backend `CORS_ORIGINS` contains exact HTTPS app origin
- Restart backend container

4. Database connection failures:

- Verify backend `.env` DB variables
- Check mysql container health and logs
- Validate credentials match compose environment

5. Tailscale access issues:

- Verify user accepted into tailnet
- Verify ACL permits destination node/port
- Validate node is online in `tailscale status`

---

19. Security Recommendations

- Rotate `SECRET_KEY`, DB password, and admin bootstrap password regularly.
- Do not expose MySQL publicly.
- Restrict access to server through Tailscale ACL tags and groups.
- Keep server OS and container images patched.
- Enforce strong password policy for all users.
- Store secrets in a managed secret system where possible.

---

20. Handover Notes

For production handover, provide DevOps with:

- Repository release/tag
- Final `backend/.env` secret values via secure channel
- Tailnet hostname and ACL policy mapping
- Backup/restore SOP owner and schedule
- Incident response contact list

End of document.